

4.3 — Enforce Structured Output Using Tool Use and JSON Schemas

Concept

Specificity beats vagueness. Examples beat instructions. Schemas beat parsing.

When you need Claude to return structured data — a parsed entity, a classification result, a scored evaluation — the default instinct is to write a prompt that says "respond in JSON with fields X, Y, Z." This is brittle. Claude will sometimes add explanatory prose, wrap the JSON in markdown fences, or omit optional fields without warning. Your downstream code then relies on regex or `JSON.parse` against free-form text that may change shape with any new model version.

The correct approach is to use `tool_use` as a structured output mechanism.

Tool Use as a Data Capture Contract

A tool does not have to do anything external. You can define a tool whose sole purpose is to receive structured data from the model. Claude fills in the parameters according to your schema; you capture the `tool_use` block from the response; the data arrives already validated against your types — no parsing, no guessing.

This pattern inverts the usual framing. Instead of "here is a function Claude can call," think "here is a typed slot Claude must fill."

`tool_choice`: Forcing a Specific Tool

By default, Claude decides whether to call a tool at all. For structured output, you must remove that choice. The `tool_choice` parameter controls this:

Value	Behavior
<code>{ "type": "auto" }</code>	Claude decides whether to use a tool (default)
<code>{ "type": "any" }</code>	Claude must call at least one tool
<code>{ "type": "tool", "name": "... " }</code>	Claude must call exactly this tool

Use `{ "type": "tool", "name": "your_capture_tool" }` to guarantee the response contains a `tool_use` block with your schema populated. The model cannot respond with free text instead.

JSON Schema Design Principles

The `input_schema` you provide is a JSON Schema object. Design it with the same care you would apply to a public API:

- **Enum constraints** — When a field has a fixed set of valid values, enumerate them. `"enum": ["low", "medium", "high"]` prevents the model from inventing values like `"moderate"` or `"elevated"`.

- **Nullable fields** — Mark optional fields explicitly. In JSON Schema, use `"type": ["string", "null"]` or set a Zod field with `.nullable()`. This signals intent: a missing value is valid and expected, not an error.
- **Required vs. optional** — List all fields you always need in the `required` array. Fields absent from `required` may be omitted; design your downstream code accordingly.
- **Descriptions on every field** — Each property description is read by the model when deciding what to populate. Treat them as contracts: specify format, units, and valid ranges inline.

Why This Beats Regex and Free-Form Parsing

- The response structure is enforced by the API, not by prompt instructions that can drift.
- Type safety flows directly from the schema to your TypeScript types via Zod inference.
- Validation happens before your code touches the data — malformed output is an API error, not a runtime exception buried in business logic.
- The schema is version-controlled alongside your code, making structural changes explicit and reviewable.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";
import { z } from "zod";
import { zodToJsonSchema } from "zod-to-json-schema";

const client = new Anthropic();

// Define the output shape with Zod — this becomes the tool's input_schema
const SentimentResultSchema = z.object({
  sentiment: z
    .enum(["positive", "negative", "neutral"])
    .describe("Overall sentiment classification of the text"),
  confidence: z
    .number()
    .min(0)
    .max(1)
    .describe("Confidence score between 0.0 and 1.0"),
  summary: z
    .string()
    .nullable()
    .describe(
      "One-sentence summary of the key sentiment signal, or null if no clear signal"
    ),
  topics: z
    .array(z.string())
    .describe("List of topics mentioned that influenced the classification"),
});

type SentimentResult = z.infer<typeof SentimentResultSchema>;
```

```

// The tool acts as a typed capture slot – no external execution needed
const captureTool: Anthropic.Tool = {
  name: "capture_sentiment",
  description:
    "Record the structured sentiment analysis result for the provided text.",
  input_schema: zodToJsonSchema(SentimentResultSchema) as Anthropic.Tool["input_schema"],
};

async function analyzeSentiment(text: string): Promise<SentimentResult> {
  const response = await client.messages.create({
    model: "claude-opus-4-5",
    max_tokens: 1024,
    tools: [captureTool],
    // Force the model to populate the schema – no free-text fallback
    tool_choice: { type: "tool", name: "capture_sentiment" },
    messages: [
      {
        role: "user",
        content: `Analyze the sentiment of the following text:\n\n${text}`,
      },
    ],
  });

  // The tool_use block is guaranteed by tool_choice – safe to assert
  const toolUseBlock = response.content.find(
    (block): block is Anthropic.ToolUseBlock => block.type === "tool_use"
  );

  if (!toolUseBlock) {
    throw new Error("Expected tool_use block not found in response");
  }

  // Validate the populated schema against our Zod definition
  const parsed = SentimentResultSchema.safeParse(toolUseBlock.input);
  if (!parsed.success) {
    throw new Error(`Schema validation failed: ${parsed.error.message}`);
  }

  return parsed.data;
}

// Usage
const result = await analyzeSentiment(
  "The new release has some rough edges but the core experience is genuinely delightful."
);

```

```
console.log(result.sentiment); // "positive"
console.log(result.confidence); // e.g. 0.78
console.log(result.summary); // "Positive overall despite minor friction"
console.log(result.topics); // ["product release", "user experience"]
```

The `zodToJsonSchema` conversion bridges Zod's type system to the JSON Schema format the Anthropic API expects. The result is a single source of truth: the Zod schema defines both the runtime validation and the wire format Claude receives.

Common Mistakes

1. **Using `tool_choice: "auto"` for structured output** — Without forcing the specific tool, Claude may respond with plain text when the answer seems obvious. Always use `{ "type": "tool", "name": "..."` `}` when you need guaranteed structure.
2. **Forgetting nullable on optional fields** — Marking a field as not required in the schema but asserting its presence in code. If the model omits an optional field, your code breaks. Use `.nullable()` in Zod and handle null explicitly.
3. **Using overly broad string types for constrained values** — A field typed as `string` when you only accept `"draft" | "published" | "archived"` will eventually receive unexpected values. Use `z.enum()` to make the constraint explicit to the model and enforced at validation.
4. **Skipping schema validation after capture** — Trusting that `tool_use.input` always matches your schema because you defined it. Model behavior can vary; always run `safeParse` and handle failures rather than assuming structural correctness.
5. **Writing "respond in JSON" in the system prompt alongside a schema** — Redundant instructions add noise and can create conflicting expectations. Let the schema and `tool_choice` do the work; remove prose instructions that duplicate what the schema already enforces.
6. **Defining a single monolithic capture tool for multiple output types** — If you need different structures depending on context, define separate tools and select between them with `tool_choice`. A single tool with a union schema forces complex conditional logic into both the schema and your code.

References

- [Anthropic: Tool use overview](#)
- [Anthropic: Forcing tool use with `tool\_choice`](#)
- [Anthropic: JSON Schema for tool inputs](#)
- [zod-to-json-schema: npm package](#)